React Context API — Step-by-Step Guide

The React Context API allows you to share data between components without prop drilling. Below is a complete guide to creating and using context.

1. Create a Context

```
import { createContext } from "react";

const MyContext = createContext();
export default MyContext;
```

2. Create a Provider

```
import React, { useState } from "react";
import MyContext from "./MyContext";

const MyProvider = ({ children }) => {
  const [user, setUser] = useState("John Doe");

  return (
    <MyContext.Provider value={{ user, setUser }}>
      {children}
    </MyContext.Provider>
  );
};

export default MyProvider;
```

3. Wrap the App with Provider

```
import MyProvider from "./MyProvider";
```

```
function App() {
  return (
    <MyProvider>
      <Home />
    </MyProvider>
  );
}

export default App;
```

4. Consume Context

```
import React, { useContext } from "react";
import MyContext from "./MyContext";

const UserProfile = () => {
  const { user, setUser } = useContext(MyContext);

  return (
    <div>
      <h1>User: {user}</h1>
      <button onClick={() => setUser("Jane Smith")}>
        Change User
      </button>
    </div>
  );
};

export default UserProfile;
```

React useReducer — Step-by-Step Guide

The `useReducer` hook is useful for more complex state logic or when updates depend on

previous state.

1. Initial State

```
const initialState = { count: 0 };
```

2. Create Reducer

```
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "reset":
      return initialState;
    default:
      return state;
  }
}
```

3. Use useReducer in Component

```
const [state, dispatch] = useReducer(reducer, initialState);
```

4. Dispatch Actions

```
<button onClick={() => dispatch({ type: "increment" })}>+</button>
<button onClick={() => dispatch({ type: "decrement" })}>-</button>
<button onClick={() => dispatch({ type: "reset" })}>Reset</button>
```

Full Example

```
import React, { useReducer } from "react";

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "reset":
      return initialState;
    default:
      return state;
  }
}

export default function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <h1>Count: {state.count}</h1>

      <button onClick={() => dispatch({ type: "increment" })}>+</button>
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </div>
  );
}
```

Context + useReducer (Bonus)

You can combine both to create a global state manager similar to Redux.